

A Pull-Request-Driven DevOps Pipeline for a Serverless Application

Alexandru Gânju (ganju@kth.se), Robert Fedus (fedus@kth.se)

Repository: <https://github.com/Gabjea/devops-project>

1 Introduction

The application we used for demonstrating this DevOps pipeline is a small, serverless URL shortener. It consists of a Python AWS Lambda function sitting behind API Gateway. It stores links in DynamoDB. We wished this application to be simple because the main focus is around the pipeline. Every change to the app or infrastructure will go through the same pull request-driven workflow. The change is tested by a CI pipeline that is planned and deployed by Atlantis. It is also verified by a smoke test and merged automatically. All AWS services run locally in Floci, an open-source AWS emulator. This enabled us to not need a cloud account.

2 Architecture and processes

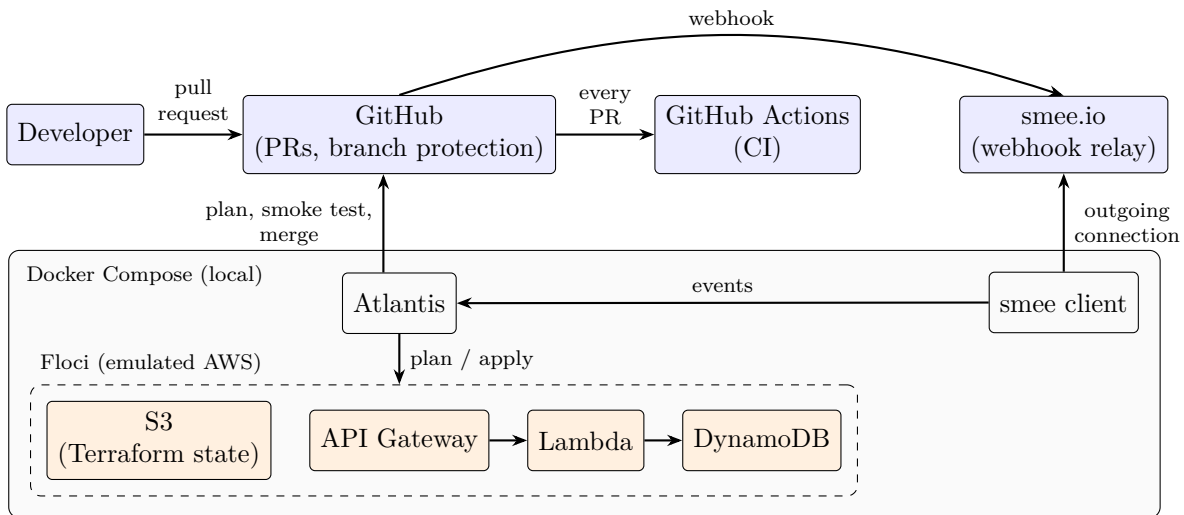


Figure 1: Components and how they interact.

Table 1 maps each required feature to its implementation.

Feature	Implementation
CI	GitHub Actions on every pull request: unit tests, <code>terraform fmt/validate</code> , Checkov, pip-audit, and integration tests that deploy the full stack into an ephemeral Floci instance and call the live API.
CD	Atlantis posts the Terraform plan on the pull request; <code>atlantis apply</code> deploys to the persistent environment, runs a smoke test, and merges the pull request.
Infrastructure as Code	Terraform for all AWS resources, with state in an S3 bucket in Floci; Docker Compose for Floci, Atlantis and the webhook relay.
Development platform	GitHub: pull requests, branch protection on <code>main</code> , required status checks.
Quality / security	Checkov, pip-audit, Dependabot, secret scanning with push protection, least-privilege IAM and tokens, pinned tool and image versions.

Table 1: Required features and how they are implemented.

What happens on a code change. First, a developer opens a pull request. The CI runs all the fast checks (unit tests, Terraform checks, security scans). If they pass, the integration test job starts Floci

and applies the Terraform code and runs the HTTP tests against the deployed API before discarding the environment. At the same time, in parallel, Atlantis plans the change against the persistent environment and posts the plan as a comment in the PR, so the reviewer will see the resources that will be created, changed, or destroyed. The branch protection requires all CI jobs and the Atlantis plan to pass. The reviewer will then comment `atlantis apply`, and Atlantis applies the plan and runs `scripts/smoke_test.sh` (creates a link, follows it, expects a 302 redirect) and merges the pull request. A rollback is a reverted pull request that goes through the same path. The application code follows the same path as the infrastructure: Terraform packages the Lambda code, and Atlantis starts planning whenever files in `app/src` change.

3 How the components interact

GitHub to Atlantis. Atlantis works with GitHub webhooks, but GitHub's servers are only able to deliver webhooks to a publicly reachable URL. Atlantis runs locally, and a webhook URL such as `http://localhost:4141` would point GitHub to its own server, so GitHub cannot reach Atlantis directly. In order to fix this, we used smee.io as a relay. GitHub now sends the webhook to a public smee channel, and a smee client in our Compose setup keeps an *outgoing* connection to that channel and forwards each event to Atlantis. Because the channel URL is public, every webhook is signed with a shared secret and Atlantis rejects unsigned requests. It also only accepts events for our repository.

Atlantis to Floci. Inside Docker Compose, `localhost` refers to the container itself, so Atlantis reaches Floci at `http://floci:4566`. The AWS endpoint is a Terraform variable (default `localhost:4566` for developers, overridden inside Atlantis). Backend blocks don't use variables, so Atlantis initialises Terraform with a small `-backend-config` file that points the S3 backend to the same address.

Floci to Lambda. Floci runs each Lambda call in a real Docker container and needs the Docker socket; it attaches these containers to the Compose network and injects the endpoint, so the function's boto3 client reaches DynamoDB in Floci without code changes.

Shared state. The Terraform state stays in a versioned S3 bucket in Floci with lock files. Because of this, developers and Atlantis that work on the same state cannot apply at the same time. Floci persists data to disk, so the environment will still be there on restarts.

4 Key design decisions

Floci instead of a cloud account. The emulator makes the project free and reproducible by anyone, since a cloud account is not required. It lets CI create a complete environment per pull request in a matter of seconds/minutes. The Terraform code uses the standard AWS provider, so the same code could target real AWS by removing the endpoint overrides.

Atlantis instead of applying from CI. A CI job that applies after merge is the simpler version. However, a failed apply then leaves `main` describing infrastructure that does not exist. Atlantis applies *before* merging, so `main` always matches what is deployed, and the plan is visible in the pull request where the review happens. It also keeps the credentials to the environment out of GitHub. Atlantis runs along with Floci and only receives events, while CI never needs access to the persistent environment.

Ephemeral environments in CI. Integration tests run against a new environment built from the same Compose file and Terraform code as the production. This tests the infrastructure code, not only the application, and leaves nothing behind. Fast checks run first so that the slower job only runs for changes that already look correct.

smee.io instead of a tunnel or a self-hosted runner. A tunnel such as ngrok gives Atlantis a public URL but it needs an account and will expose the service directly. A self-hosted GitHub runner avoids webhooks but would replace Atlantis's plan comments, locking and applying requirements. smee needs no account and no inbound connection, which made it the easiest and simplest option for running locally.

5 Limitations and trade-offs

- **Emulator fidelity.** Floci is not AWS, so the limits, IAM evaluation and service behaviour may differ, so the passing tests will not guarantee that the same code works perfectly in AWS.
- **Single machine.** Atlantis and the persistent environment only run while the local machine (like a laptop) is on, and smee.io is a development relay, not meant for actual, production webhooks.
- **No persistent staging.** Ephemeral environments are always empty on startup, so they can't catch problems that only appear when updating existing resources, like a change that replaces a table and loses its data. However, the Atlantis plan mitigates this, because it shows the replacements before anything is applied.
- **Delivery, not deployment.** A human triggers every apply. This adds a concrete review of the plan but will slow releases, and it depends on someone being available. A failed smoke test reports the problem but does not roll back automatically.
- **Approval rule.** The current setup was designed to require an approving review before applying, but GitHub does not allow approving one's own pull request. While working alone, we relaxed this rule, which removes the two-person check for changes.

6 Conclusion

The project used CI, CD, IaC, and security automation in one single repository, where every change is tested in a temporary environment. The changes are reviewed as a Terraform plan, and deployed by Atlantis and verified by a smoke test. The trade-offs identified are the fact that we use an emulator instead of real AWS account, and a single-machine CD setup. But because we chose this architecture, we kept the project free and easy to reproduce.

7 Use of AI-assisted tools

We used LLMs for language assistance when creating the README file in the repository, and for troubleshooting our work when we encountered errors.